YARA Cheat Sheet

Runtime Options

- `-r` : Recursive scanning (scans directories and subdirectories).
- `-s` : Display matched strings and their offsets.
- `-D` : Display module data (useful for debugging).

Rule Modifiers

- `global` : Ensures this rule is evaluated before other rules.
- `private` : Prevents the rule from producing output when it hits (useful for dependency rules).
- `rule_name` (in conditions): Allows you to use the boolean outcome of another rule within your condition logic (e.g., `condition: $a and MyOtherRule`).

String Definitions

String Types

- **Text Strings:** `""`
  - Represents ASCII or Wide (UTF-16) strings.
  - **Modifiers:** `ascii`, `wide`, `fullword`, `base64`, `xor`
- **Hex Strings:** `{}`
  - Represents raw hexadecimal bytes.
  - **Wildcards:** `??` (any byte)
  - **Jumps:** `[4-8]` (jump 4 to 8 bytes)
  - **Alternatives:** `( 01 02 | 03 04 )` (match `01 02` OR `03 04`)
- **Regular Expressions:** `/ /`
  - Standard regex syntax, enclosed in forward slashes.

String Symbols (in Conditions)

- `$`: Represents a string (e.g., `$a`, `$hex_string`).
- `@`: Represents the offset (address) of a string match (e.g., `@a[1] > 100`).
- `#`: Represents the count (number of occurrences) of a string (e.g., `#a == 5`).

Conditions

Standard boolean logic and keywords:

- **Operators:** `and`, `or`, `not`

- **Grouping:**
  - `(any of them)`: Shorthand for `($a or $b or $c)`.
  - `(all of them)`: Shorthand for T `($a and $b and $c)`.
  - `(2 of ($str*))`: True if at least 2 strings starting with `$str` are found.
- **File Size:** `filesize > 100KB` or `filesize < 1MB`.

Loops

Used to iterate over string matches or module data.

```
for <number> <variable> in <iterable_function>:
    ( <variable>.<property> <condition> )
```

Example: `for any i in (1..#a) : ( @a[i] < 100 )` *(any match of `$a` is found in the first 100 bytes).*

File Headers / Magic Bytes

Use `uintXXyy(offset)` to read integers from a file's offset.

- **`uint`**: Unsigned integer.
- **`XX`**: Length in bits (e.g., `16` or `32`).
- **`yy`**: Endianness. `le` (little-endian) is default. `be` is for big-endian.

Examples:

- **Windows PE:** `uint16be(0) == 0x4D5A` *(Reads "MZ" at offset 0).*
- **LNK Header:** `{ 4c 00 00 00 01 14 02 00 } at 0`
- **ELF Header:** `uint32be(0) == 0x7F454C46` *(Reads "\x7FELF" at offset 0).*
- **OOXML (Zip):** `{ 50 4B 03 04 14 00 06 00 } at 0` *(Reads "PK\x03\x04...")*

PE Module

(Requires `import "pe"`)

Languages

- `pe.locale(0x0419)` *// Russian (RU)*
- `pe.locale(0x0004)` *// Chinese (CN)*
- `pe.locale(0x0412)` *// Korean (KR)*

Imports & Exports

- `pe.imphash()`: Returns the import hash.
- `pe.imports("kernel32.dll", "CreateFileA")`: Checks for a specific function import.
- `pe.import_details`: Use in a loop to check import properties.

- `pe.exports("MyFunctionName")`: Checks for a specific exported function.

PDB Path

- `pe.pdb_path`: Accesses the PDB (Program Database) path string.
  - **Useful with modifiers:** `pe.pdb_path contains "C:\\Users\\"`

File Metadata (Version Info)

- `pe.version_info["field_name"]`
- **Common Fields:**
  - `CompanyName`
  - `FileDescription`
  - `InternalName`
  - `LegalCopyright`
  - `LegalTrademarks`
  - `OriginalFilename`
  - `ProductName`

Certificates (Signatures)

- `pe.signatures[0].serial`: Serial number (usually `index = 0`).
- `pe.signatures[0].thumbprint`: Thumbprint (usually `index =0`).

- `pe.signatures[0].subject`: Subject string (usually `index = 0`).
  - **Useful with modifiers:** `pe.signatures[0].subject contains "Microsoft"`

Console Module

(Requires `import "console"`)

- `console.log("Debug message")`
- `console.log(pe.imphash())`: Very useful for printing module data during rule testing.

Common Modifiers (for Modules)

These functions can be used on string properties from modules (like PDB path or certificate subject):

- `startswith`
- `endswith`
- `contains`